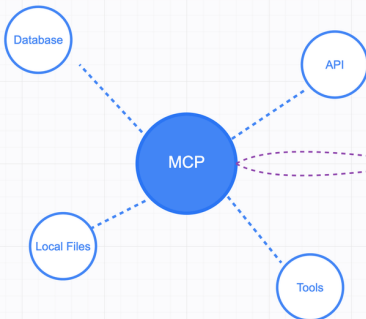


# 2

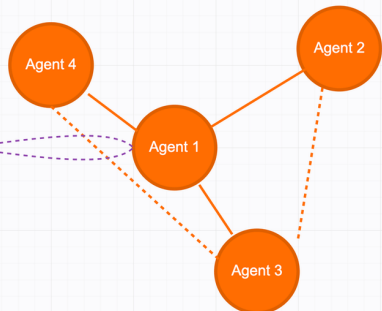
## AGENT PROTOCOLS

Every AI Engineer Should Know

MCP vs A2A: Comparing AI Agent Protocols



**Model Context Protocol**  
Connecting AI to data sources



**Agent-to-Agent Protocol**  
Enabling agent collaboration



AI Coach John

Follow



PROITBRIDGE  
Shaping the Future of AI

Repost

# Two Questions, Not Two Rivals

Most confusion comes from treating these as competitors. **They operate at different layers of an agentic system, and production stacks increasingly run both together.**

## MCP ASKS

How does an agent talk to **tools, data and APIs**?

## A2A ASKS

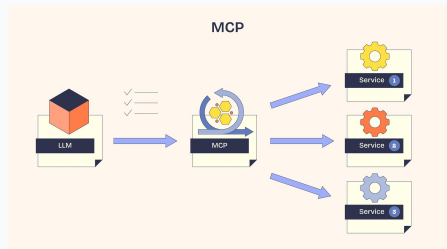
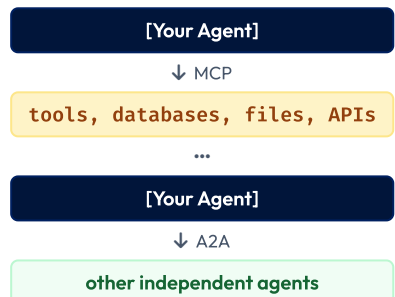
How do independent agents talk to **each other**?

## REAL SCENARIO

Your booking agent must query an internal price database *and* hand the actual reservation to a partner company's agent.

Two connections. Two protocols. One system.

## THE TWO CONNECTIONS



**You own your tools.** You do not own another company's agent. That single fact drives every difference between the two protocols.



# The Shared Foundation

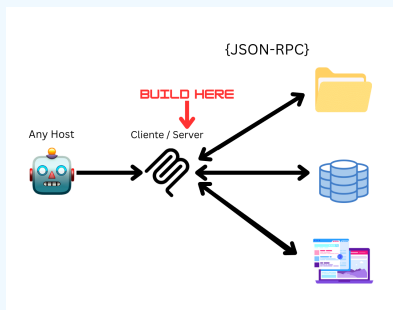
## WHAT IT IS

A lightweight protocol for calling a procedure on a remote system and getting a result back, using plain JSON messages. **Both MCP and A2A use it as their wire format.**

## THREE FIELDS

- > **method:** what to call
- > **params:** the arguments
- > **id:** pairs a reply to its request

A versionable, well established format. Neither protocol invented it, which is why both can add capabilities without breaking older clients.



## </> ON THE WIRE

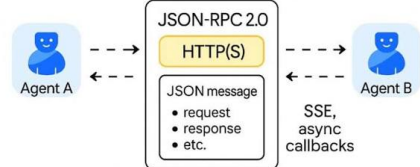
### ↑ REQUEST

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {"city": "Tokyo"}
  },
  "id": 1
}
```

### ↓ RESPONSE

```
{
  "jsonrpc": "2.0",
  "result": {
    "temperature": "22C",
    "condition": "cloudy"
  },
  "id": 1
}
```

## A2A protocol



# MCP (Model Context Protocol)

## CONCEPT

**What:** A standard interface between an AI agent and external tools, data and services.

**Why:** One consistent wiring instead of a custom integration per tool.

**When:** Any time your agent must call tools, query data, or read files.

**How:** Client connects → server declares capabilities → agent calls them.

## SCENARIO

Created by Anthropic, MCP is the interface between an AI's brain and its hands.

You want your agent to read Postgres, search Drive and hit an internal API. Without MCP that is three bespoke integrations, rebuilt for every AI provider you support.

## ARCHITECTURE

[MCP Client] AI agent or host

↓ JSON-RPC 2.0

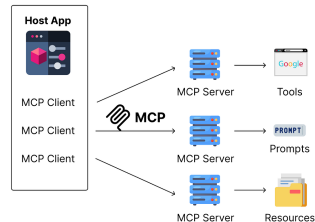
stdio (local) or HTTP  
(remote)

↓

[MCP Server] tool provider

↓ exposes

tools, resources, prompts



## COMMON MISTAKE

MCP does not decide **when** a tool runs. That stays with your agent framework. MCP only supplies the wiring.



# MCP in 2026

**97M**

monthly SDK downloads

**9,000+**

community MCP servers

**Dec 2025**

donated to Linux Foundation

## "1" TRANSPORT

JSON-RPC 2.0 over **stdio** for local servers, or **Streamable HTTP** for remote ones.

## ⚖️ GOVERNANCE

Donated by Anthropic to the Linux Foundation's **Agentic AI Foundation** in December 2025.

Now a neutral, vendor independent standard rather than one company's format.

## 🌐 NATIVE SUPPORT



Claude



ChatGPT



Gemini



Cursor



Windsurf



JetBrains

**Why the scale matters.** Most tools you would want to connect already have an MCP server, so you are wiring rather than building. Figures as of February 2026.



AI Coach John

[Follow](#)PROITBRIDGE  
Build For Better Futures[↻ Repost](#)

# MCP in Code

## 📘 WHAT THIS DOES

Defines a single tool an MCP server exposes. Any compatible client can then discover and call it through the same interface.

## ✗ WITHOUT MCP

A custom integration per tool, per client. The work multiplies as **N x M**.

## ✓ WITH MCP

One small server file. Every compatible client can already use it. The work collapses to **N + M**.

## </> WEATHER\_SERVER.PY

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather-server")

@mcp.tool()
def get_weather(city: str) → str:
    """Get current weather for a city."""
    # your weather API call here
    return f"{city}: 22C, cloudy."

if __name__ == "__main__":
    mcp.run()
```

**Deliberately minimal.** Wrapping an existing API takes a decorator and a small server file, not a bespoke build per AI provider.



# A2A (Agent2Agent Protocol)

## CONCEPT

**What:** An open standard letting independent agents discover, communicate and collaborate.  
**Why:** Agents built by different companies need a common language.  
**When:** Delegating real work to an agent you do not control.  
**How:** Read its Agent Card → submit a Task → collect Artifacts.

## SCENARIO

Introduced by Google in April 2025.  
 Think of it as HTTP for AI agents.

Your assistant needs a hotel booked. A partner company runs a specialist booking agent. A2A is how yours finds it, delegates to it, and tracks the job to completion.

## ARCHITECTURE

[Client Agent] delegates work

↓ JSON-RPC 2.0 over HTTP

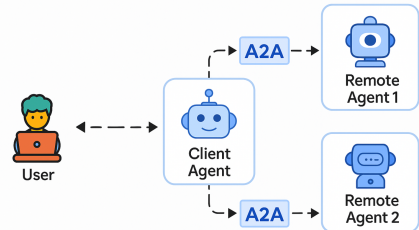
+ SSE, webhooks for async

↓

[Remote Agent] executes it

↓ returns

Messages, then Artifacts



## TERMINOLOGY

It is **client and remote**, not client and server. Both sides are autonomous agents, not a caller and a passive service.



# Agent Cards

## WHAT IT IS

A JSON document every A2A agent publishes, typically at `/.well-known/agent-card.json`, describing its capabilities, supported input and output types, and authentication requirements.

## WHAT IT ENABLES

- **Discover:** find an agent with no prior integration
- **Evaluate:** check skills and auth before trusting it
- **Verify:** signed cards prove the card is genuine

This is how a client agent sizes up a remote agent **before** handing over any work.

## AGENT-CARD.JSON

```
{
  "name": "TravelBookingAgent",
  "description": "Books flights and
    hotels for a client agent",
  "capabilities": [
    "flight_search",
    "hotel_booking"
  ],
  "authentication": {
    "schemes": ["OAuth2"]
  }
}
```

**A business card for machines.** Critical when the two agents belong to entirely different organisations.



# The Task Lifecycle



## EVERY TASK HAS AN ID

Work is addressable, so a client agent can poll it, resume it or cancel it long after the first call returns.



## BUILT FOR LONG JOBS

A task can persist, pause for input, and be tracked across a collaboration running for hours.



## WHY IT MATTERS

This makes A2A meaningfully more **stateful** than an MCP tool call.

A tool call returns a value and is finished. A task has a life you can observe, interrupt and resume.



## STATE MACHINE

submitted



working



completed



## BRANCHES FROM WORKING

input-required

remote agent needs more info

failed

execution could not complete

canceled

stopped by the client agent



# Messages and Artifacts

## MESSAGES

The back and forth **while a task is in progress**: status updates, clarifying questions, intermediate results.

## ARTIFACTS

The deliverables. The **final outputs** the remote agent produces once the task reaches completed.

## WHY SEPARATE THEM

So a client agent can track progress without confusing *"here is an update"* with *"here is the result you asked for."*

## </> A MESSAGE IN FLIGHT

```
{
  "role": "agent",
  "parts": [{
    "type": "text",
    "text": "Found 3 flight
            options under $500."
  }]
}
```

## ACROSS ONE TASK

"Still searching" (Message)



"3 options found" (Message)



Booking file (Artifact)



# A2A in 2026

**50+**

launch partners

**100+**

partners today

**v0.3**

July 2025 release

## TRANSPORT

JSON-RPC 2.0 over **HTTP(S)**, plus gRPC and REST/JSON. Server-Sent Events and webhooks carry long running or async tasks.

## GOVERNANCE

Also now under the **Linux Foundation**, following Google's original April 2025 release.

Version 0.3 added gRPC support and signed security cards.

## PARTNERS



Salesforce



SAP



ServiceNow



LangChain



PayPal

**The signal that matters.** Both protocols now sit under Linux Foundation governance. Neither is a single vendor lock-in play, which is much of why enterprise adoption accelerated.



AI Coach John

[Follow](#)PROITBRIDGE  
Open For Better Future Repost

# MCP vs A2A Cheatsheet

|                     |  MCP |  A2A |
|---------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Solves</b>       | Agent to tool connection                                                              | Agent to agent collaboration                                                          |
| <b>Architecture</b> | Client and server                                                                     | Client and remote, both autonomous                                                    |
| <b>Transport</b>    | stdio or Streamable HTTP                                                              | HTTP(S), gRPC or REST/JSON                                                            |
| <b>Discovery</b>    | Host app exposes tools                                                                | Agent Cards at well known URLs                                                        |
| <b>State</b>        | Largely stateless tool calls                                                          | Stateful, explicit task lifecycle                                                     |
| <b>Core unit</b>    | Tool call                                                                             | Task, with Messages and Artifacts                                                     |
| <b>Governance</b>   | Linux Foundation, Dec 2025                                                            | Linux Foundation, from Google                                                         |
| <b>Analogy</b>      | Interface to an AI's tools                                                            | HTTP for agent to agent traffic                                                       |

**Read the rows, not the columns.**

MCP calls something that **returns a value**. A2A hands work to something that **makes decisions**.



AI Coach John

Follow



PROITBRIDGE  
Done For Better Future

 Repost

# They Stack, They Do Not Replace

## THE PATTERN

A single agent commonly uses **MCP** to reach its own tools and data, and **A2A** to delegate work to a completely separate agent.

## ONE LAYER DOWN

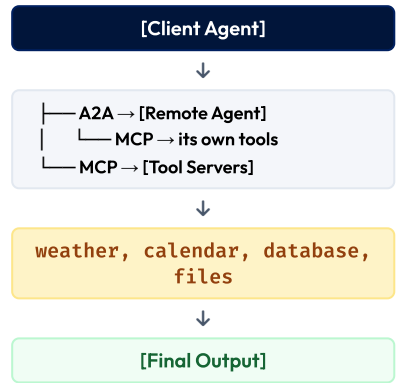
That remote agent, built by another team or company, almost certainly uses MCP internally for its own tool access.

Same wiring pattern, repeated at every level of the system.

## NOTE

An MCP path ends at a tool that returns a value. It does not delegate further.

## LAYERED ARCHITECTURE



**They stack, they do not substitute.**  
Every non-trivial agentic system ends up using both.



# Security for Both

## A2A

Layer **OAuth 2.0** for mutual agent authentication and **JWTs** for signed, verifiable identity.

Critical because A2A often crosses organisational boundaries where you cannot trust the other side by default.

## MCP

Scope permissions **per tool**.

A tool with database write access should not be callable by every agent that connects, and read-only must be enforced at the credential level, not by prompt instruction.

## BOTH PROTOCOLS

Use **signed capability declarations**:

- > Signed Agent Cards in A2A
- > Verified tool manifests in MCP

A client should be able to prove a discovered capability is what it claims to be, not a spoofed one.

## SHARED FAILURE MODE

A capability you never verified, called with a credential that was **scoped too widely**.



# When To Use Which

## USE MCP WHEN

- > Your agent needs to call tools, query databases, read files or hit APIs
- > You are building a single agent, or a tightly coupled internal system, that just needs consistent tool access

## USE A2A WHEN

- > Your agent delegates to a genuinely separate, autonomous agent, especially one built by another team or company
- > You need formal task tracking across a longer running collaboration
- > Discovery matters and you do not know in advance which agent you will work with

## USE BOTH WHEN

You are building any non-trivial multi-agent system.

**MCP** for each agent's own tool access.  
**A2A** for how separate agents hand off work to each other.

This is the default shape of a real production stack, not an advanced option.

## THE QUICK TEST

Does the other end **return a value** or **make decisions**? That answers it.



# Common Misconceptions

## ✖ "THEY ARE COMPETING STANDARDS"

False. They operate at different layers, agent to tool versus agent to agent, and were **designed to complement** each other.

## ✖ "BOTH INVENTED THEIR OWN FORMAT"

False. Both are built on **JSON-RPC 2.0**, which is exactly why they can evolve without breaking existing integrations.

## ✖ "A2A REPLACES MCP"

False. A remote agent reached over A2A **almost certainly still uses MCP** internally to reach its own tools.

## ✖ "PICK ONE FOR THE WHOLE SYSTEM"

False. Most production multi-agent systems use **both simultaneously**, in different parts of the architecture.



# One Wiring Layer, **One Collaboration Layer**



## MCP

how an agent reaches its tools



## A2A

how agents reach each other

Both speak **JSON-RPC 2.0** underneath. Both are now governed by the **Linux Foundation**. Both are increasingly used together in real production systems.

It was never a choice between them.

 **Save this as your protocol reference sheet.**

Follow AI Coach John for more AI engineering breakdowns.



AI Coach John

[Follow](#)



**PROITBRIDGE**  
Open For Better Futures

 **Repost**